

FACULTY OF ENGINEERING AND BASIC SCIENCES
ACADEMIC PROGRAM: DATA ENGINEERING AND ARTIFICIAL INTELLIGENCE
COURSE: ETL (G01)

Lab 1B – From Business Requirements to a Basic ETL Pipeline

1. Introduction

In Lab 1A, your team identified the business objectives, analytical questions, data requirements, KPIs, and expected outputs for a retail analytics platform. In this laboratory, you will build the first working version of the data system required to satisfy those needs.

The focus is not on advanced transformations or optimization. The focus is on systems thinking: designing the pipeline as a set of connected blocks, implementing each block independently, and verifying that all components work together as one end-to-end system.

In real-world Data Engineering projects, technical implementation is never the first step. Before designing databases, writing code, or building ETL pipelines, engineers must first understand the business problem they are trying to solve.

2. Learning Objectives

Implement a basic ETL pipeline that integrates heterogeneous retail data sources and produces a structured analytical repository capable of answering the business questions defined in Lab 1A.

By the end of this lab, students will be able to:

- Translate business requirements into technical pipeline components.
- Represent an ETL system using a block diagram before coding.
- Extract data from CSV, JSON, and XML sources.
- Perform basic profiling, cleaning, schema harmonization, integration, and transformation.
- Load processed data into a SQLite database.
- Execute analytical SQL queries that verify whether the business requirements were satisfied.
- Explain how each block contributes to the behavior of the complete system.

3. Dataset Description

The data cover three months (February–April 2026) and three branches. Transaction data are intentionally distributed across different formats to simulate heterogeneous operational systems.

File	Format	Purpose
sales_cali.csv	CSV	Sales transactions from Cali.
sales_bogota.json	JSON	Sales transactions from Bogotá.
sales_medellin.xml	XML	Sales transactions from Medellín.

File	Format	Purpose
products.csv	CSV	Product names, categories, list prices, and costs.
stores.csv	CSV	Store, city, and region information.
promotions.csv	CSV	Promotion dates, discount percentages, and campaigns.
monthly_targets.csv	CSV	Monthly sales target for each branch.

The transaction sources contain a small number of realistic quality issues. Students are expected to discover them through profiling rather than receiving a predefined list.

4. Dataset Description

Before implementing code, represent the solution using the following blocks:

1. Business requirements
2. Data sources
3. Extract
4. Profile
5. Clean and harmonize
6. Transform and integrate
7. Validate
8. Load
9. Query and evaluate business requirements

Your diagram must show the input and output of every block and the connection between blocks.

Do not begin coding until the team can explain how the complete system should work.

5. Recommended Project Structure

```

Lab1B_ETL/
|
├── data/
|   ├── raw/
|   ├── processed/
|   └── output/
|
├── database/
|   └── retail_analytics.db
|
├── src/
|   ├── extract.py
|   ├── transform.py
|   ├── load.py
|   ├── queries.py
|   └── main.py
|

```

```

|
|— docs/
|   |— pipeline_diagram.png
|
|— README.md
|— requirements.txt
|— .gitignore

```

6. Activities

Activity 1 – Review and Trace the Requirements

Review your Lab 1A deliverable and select the six highest-priority business requirements that will be implemented in the first version of your analytical platform.

For each requirement, identify the data needed, the expected transformation, and the expected analytical output. These requirements will guide the design and implementation of your ETL pipeline.

Business Requirement	Required Data	Pipeline Block	Expected Output

Activity 2 – Design the Pipeline

Create a block diagram showing all data sources, processing stages, outputs, and connections. For each block, define: purpose, input, processing responsibility, output, and possible failure.

Block	Input	Responsibility	Output	Possible Failure
Extract				
Profile				
Clean/Harmonize				
Transform/Integrate				
Validate				
Load				
Query				

Activity 3 – Extract

Implement one extraction function for each transaction format and functions for the reference tables. Every transaction source must be converted to a pandas DataFrame with a common schema.

Common Field	Meaning
sale_line_id	Unique sale-line identifier
sale_date	Transaction date
store_id	Store identifier
product_id	Product identifier
quantity	Units sold
unit_price	Price before discount
promotion_code	Applied promotion, when applicable
payment_method	Payment method

The extraction block must not perform business calculations. Its responsibility is to read each source and expose the data in a common technical structure. No cleaning, validation, or business transformations should be performed during the extraction stage.

Activity 4 – Profile the Extracted Data

Before cleaning or transforming, generate a compact profiling summary for the combined transaction data: row count, columns and data types, missing values, duplicate sale_line_id values, invalid quantities, invalid prices, invalid dates, and distinct values for selected categorical fields.

Record the findings in the README and explain which cleaning decisions are derived from each finding.

Activity 5 – Clean and Harmonize

Cleaning improves data quality. Apply only the rules supported by the profiling findings:

- Standardize column names and ID values.
- Trim whitespace and standardize text casing.
- Parse the different date formats into one date type.
- Convert quantity and unit_price to numeric values.
- Remove duplicated sale_line_id records, keeping the first valid occurrence.
- Reject records with invalid dates, quantity ≤ 0 , or unit_price ≤ 0 .
- Represent missing promotion codes consistently.

All cleaning decisions must be justified using the profiling results obtained in Activity 4.

Activity 6 – Transform and Integrate

Transformation changes data so that it can satisfy analytical requirements. Integrate the transactions with the reference tables and create:

- product_name and category from products.csv
- store_name, city, and region from stores.csv
- discount_pct and campaign_name from promotions.csv
- gross_sales = quantity $\times$ unit_price

- $\text{discount_amount} = \text{gross_sales} \times \text{discount_pct}$
- $\text{net_sales} = \text{gross_sales} - \text{discount_amount}$
- month, week, and day_name from sale_date
- sales_target from monthly_targets.csv

If you considered, you could add other columns to the integrated sales table.
Only transformations required to satisfy the selected business requirements should be implemented.

Activity 7 – Validate

Before loading, verify at minimum:

- sale_line_id is unique.
- Required identifiers and dates are not null.
- quantity, unit_price, gross_sales, and net_sales are positive.
- Every product matches the product master.
- Every store matches the store master.
- net_sales equals gross_sales minus discount_amount.

If a critical validation fails, the pipeline should stop before loading and record the reason in the log.
Validation is the final quality gate before loading the analytical repository.

Activity 8 – Load

Save the processed dataset as data/processed/integrated_sales.csv and load it into a SQLite database named retail_analytics.db using a table named sales_analytics. The database is the analytical output of the pipeline; the final queries must read from SQLite, not from the raw files.

Activity 9 – Analytical Queries

Implement a simple menu or script that shows selected business requirements selected.
Each query must include a short explanation connecting the result to a business question from Lab 1A.

Activity 10 – Orchestrate the Complete System

Implement main.py as the entry point. It must call the blocks in the correct order, record start/end messages and row counts, stop on critical failures, and produce the CSV, SQLite database, and log file. Run the complete pipeline from the project root using one command.

Activity 11 – Verification of Requirements

Complete the table after executing the pipeline.

Business Requirement	Evidence Produced	Satisfied?	Explanation
		Yes / No	

Business Requirement	Evidence Produced	Satisfied?	Explanation
		Yes / No	
		Yes / No	
		Yes / No	

Activity 12 – Reflection Questions

1. How did the requirements from Lab 1A influence the design of the pipeline?
2. What is the difference between profiling, cleaning, transformation, and validation in your implementation?
3. Why was it necessary to design the system as blocks before coding?
4. Which block would be most affected if a branch changed its file format?
5. Did the team build an ETL pipeline, or did it build a system to solve a business problem? Explain.

7. Deliverables

Each team must submit **a single text file (.txt) containing the GitHub repository URL through the LMS.**

The repository must contain:

- Source code organized in a clear project structure.
- The input datasets provided for the laboratory.
- The generated SQLite database.
- A **README.md** explaining the project.
- Instructions for running the ETL pipeline.
- The implemented analytical queries.
- Any additional files required to reproduce the solution.

The repository should be well organized, properly documented, and executable without requiring modifications to the source code.

README.md Requirements

The README should include:

1. Project overview.
2. System architecture (pipeline diagram).
3. Selected business requirements.
4. ETL pipeline description.
5. Project structure.
6. Execution instructions.
7. Technologies used.
8. Example analytical results.

8. Assessment Rubric

Criterion	Evidence	%
1. Requirements Traceability	The selected business requirements are clearly reflected in the implemented pipeline and analytical outputs.	15%
2. System Design	The repository includes a clear pipeline design (blocks/components) showing data flow and system organization before implementation.	15%
3. ETL Implementation	The Extract, Transform and Load stages are correctly implemented and integrated into a functional pipeline.	30%
4. Code Organization	Modular structure, readable code, appropriate naming conventions, and project organization.	10%
5. Data Loading and Analytical Queries	Data are correctly loaded into SQLite and the required analytical queries produce the expected results.	15%
6. Documentation (README)	Clear README including project overview, system architecture, execution instructions, and repository organization.	15%

Think in Systems. Build Data Solutions. Create Value.